

第2回 AIプログラミング1

変数と代入

大西朔永

変数と代入の基礎

第1章の復習

- 3つの開発環境を準備
 - Google Colaboratory (メイン環境)
 - VSCode (ローカル環境)
 - Cloud Shell Editor (ブラウザ上のVSCode)

`print()` で文字列を表示、四則演算を実行

今回は変数と代入を学ぶ

変数とは

- プログラムの中で値に名前を付けて保存する仕組み

値を直接使う場合

```
print(100)  
print(100)
```

変更時にすべて書き換え

変数を使う場合

```
price = 100  
print(price)  
print(price)
```

変数の値を変えるだけ

変数 = 値に付けた名札

代入文の書き方

代入文

変数名 · = · 値

- `=` は「等しい」ではなく「代入」（右辺の値を左辺の変数に格納）
- 変数は宣言なしでそのまま使用可能

例

```
x = 10  
name = "Python"  
pi = 3.14
```

C言語との比較：変数宣言

C言語

```
int x = 10;  
float pi = 3.14;  
char name[] = "Python";
```

- 型宣言が必要
- 宣言した型以外の値を代入不可

Python

```
x = 10  
pi = 3.14  
name = "Python"
```

- 型宣言が不要（動的型付け）
- 同じ変数に異なる型の値を代入可能

静的型付け（コンパイル時に型を決定） ⇔

動的型付け（実行時に型を決定）

変数の代入と参照

ai_programming_1_02_01.py

```
1  # 変数の代入と参照
2
3  # 変数への代入
4  x = 10
5  print(x)
6
7  # 変数の上書き
8  x = 20
9  print(x)
10
11 # 複数の変数
12 name = "Python"
13 version = 3
14 print(name)
```

実行結果

```
10↵
20↵
Python↵
3↵
```

- 変数に値を代入して `print()` で表示
- 同じ変数に再代入すると値が上書きされる

変数の代入と参照

ai_programming_1_02_01.py

```
1  # 変数の代入と参照
2
3  # 変数への代入
4  x = 10
5  print(x)
6
7  # 変数の上書き
8  x = 20
9  print(x)
10
11 # 複数の変数
12 name = "Python"
13 version = 3
14 print(name)
15 print(version)
```

実行結果

```
10↵
20↵
Python↵
3↵
```

- 変数に値を代入して `print()` で表示
- 同じ変数に再代入すると値が上書きされる

変数名のルール

使える文字

- 英字 (a-z, A-Z)
- 数字 (0-9) ※先頭は不可
- アンダースコア (`_`)

使えない名前

- 予約語 (`if`, `for`, `class` 等)
- 数字で始まる名前
- 空白を含む名前

命名規則: スネークケースを推奨

`student_name = "太郎" ... # スネークケース (推奨)`

`studentName = "太郎" ... # キャメルケース (非推奨)`

まとめ

変数

値に名前を付けて保存
宣言なしで使用可能

代入文

変数名 = 値
= は代入を意味

動的型付け

型宣言が不要
C言語との大きな違い

変数名はスネークケースで、意味のわかる名前を付ける

データ型と型変換

データ型とは

- 値の種類を区別する仕組み
- プログラムが値をどう扱うかを決定

数値

計算に使える

文字列

テキスト表示
に使える

真偽値

条件判定に使
える

型によって使える演算や操作が異なる

基本データ型の一覧

型名	説明	例
<code>int</code>	整数	<code>42</code> , <code>-7</code> , <code>0</code>
<code>float</code>	浮動小数点数	<code>3.14</code> , <code>-0.5</code> , <code>1.0</code>
<code>str</code>	文字列	<code>"Hello"</code> , <code>'Python'</code>
<code>bool</code>	真偽値	<code>True</code> , <code>False</code>

Pythonの `int` は**任意精度**（桁数に制限なし）

文字列リテラルの書き方

シングルクォート / ダブルクォート

```
s1 = 'Hello'  
s2 = "Hello"
```

トリプルクォート（複数行文字列）

```
s3 = '''1行目  
2行目  
3行目'''
```

どちらも同じ文字列

エスケープシーケンス

- `\n`: 改行
- `\t`: タブ
- `\\`: バックスラッシュ

type関数で型を確認

type関数
type(値または変数)

■ 値や変数のデータ型を返す関数 使用例

```
>>> type(42)
<class 'int'>
>>> type("Hello")
<class 'str'>
```

```
>>> x = 3.14
>>> type(x)
<class 'float'>
```

C言語との比較：データ型

C言語

型	説明
<code>char</code>	1文字（1バイト）
<code>int</code>	整数（通常4バイト）
<code>float</code>	単精度浮動小数点
<code>double</code>	倍精度浮動小数点

精度・範囲に制限あり

Python

型	説明
<code>str</code>	文字列（長さ制限なし）
<code>int</code>	整数（任意精度）
<code>float</code>	倍精度浮動小数点
<code>bool</code>	真偽値

`int` は桁数制限なし

型変換（キャスト）

型変換関数

`int(値)` · `float(値)` · `str(値)`

```
>>> int("123")
123
>>> float("3.14")
3.14
>>> str(42)
'42'
```

変換エラー

```
>>> int("abc")
ValueError: ...
```

数値に変換できない文字列
はエラー

データ型と型変換の確認

ai_programming_1_02_02.py

```
1  # データ型と型変換
2
3  # type() で型を確認
4  a = 42
5  print(type(a))
6
7  b = 3.14
8  print(type(b))
9
10 c = "Hello"
11 print(type(c))
12
13 d = True
14 print(type(d))
15
```

実行結果

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
123 <class 'int'>
123.0 <class 'float'>
42 <class 'str'>
```

- `type()` で型を確認
- `int()`, `float()`, `str()` で型変換

データ型と型変換の確認

ai_programming_1_02_02. py

```
11 print(type(c))  
12  
13 d = True  
14 print(type(d))  
15  
16 # 型変換  
17 x = "123"  
18 y = int(x)  
19 print(y, type(y))  
20  
21 z = float(x)  
22 print(z, type(z))  
23  
24 w = str(42)  
25 print(w, type(w))
```

実行結果

```
<class 'int'>  
<class 'float'>  
<class 'str'>  
<class 'bool'>  
123 <class 'int'>  
123.0 <class 'float'>  
42 <class 'str'>
```

- `type()` で型を確認
- `int()`, `float()`, `str()` で型変換

まとめ

基本データ型

int, float
str, bool

type()関数

型の確認

```
<class '型名'>
```

型変換

int(),
float(), str()

変換不可はエラー

Pythonは動的型付けだが、**型を意識することが重要**

演算子と式

算術演算子

演算子	意味	例	結果
+	加算	<code>10 + 3</code>	<code>13</code>
-	減算	<code>10 - 3</code>	<code>7</code>
*	乗算	<code>10 * 3</code>	<code>30</code>
/	除算 (小数)	<code>10 / 3</code>	<code>3.333...</code>
//	整数除算	<code>10 // 3</code>	<code>3</code>
%	剰余	<code>10 % 3</code>	<code>1</code>
**	べき乗	<code>2 ** 10</code>	<code>1024</code>

算術演算子

演算子	意味	例	結果
<code>+</code>	加算	<code>10 + 3</code>	<code>13</code>
<code>-</code>	減算	<code>10 - 3</code>	<code>7</code>
<code>*</code>	乗算	<code>10 * 3</code>	<code>30</code>
<code>/</code>	除算 (小数)	<code>10 / 3</code>	<code>3.333...</code>
<code>//</code>	整数除算	<code>10 // 3</code>	<code>3</code>
<code>%</code>	剰余		
<code>**</code>	べき乗		

`/` は常に `float` を返す (`10 / 2 → 5.0`)

整数除算と剰余

整数除算 //

```
>>> 17 // 5
3
>>> -17 // 5
-4
```

小数点以下を切り捨て
(負の無限大方向)

剰余 %

```
>>> 17 % 5
2
>>> -17 % 5
3
```

$a == (a // b) * b + (a \% b)$ が常に成立

C言語の / (整数同士) → Pythonの // に対応

演算子の優先順位

優先度	演算子	説明
高	**	べき乗
↑	+×, -×	単項プラス・マイナス
	*, /, //, %	乗除・剰余
↓	+, -	加減
低	==, !=, < 等	比較

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
```

迷ったら**括弧** `()` で明示する

複合代入演算子

演算子	意味	等価な式
<code>+=</code>	加算して代入	<code>x = x + 1</code>
<code>-=</code>	減算して代入	<code>x = x - 1</code>
<code>*=</code>	乗算して代入	<code>x = x * 2</code>
<code>/=</code>	除算して代入	<code>x = x / 2</code>

```
count = 0  
count += 1 ... # count は 1  
count += 5 ... # count は 6
```

Pythonには `++` / `--` 演算子は存在
しない

比較演算子と真偽値

演算子	意味	例	結果
<code>==</code>	等しい	<code>10 == 10</code>	<code>True</code>
<code>!=</code>	等しくない	<code>10 != 5</code>	<code>True</code>
<code><</code>	より小さい	<code>10 < 20</code>	<code>True</code>
<code>></code>	より大きい	<code>10 > 20</code>	<code>False</code>
<code><=</code>	以下	<code>10 <= 10</code>	<code>True</code>
<code>>=</code>	以上	<code>10 >= 20</code>	<code>False</code>

比較演算子と真偽値

演算子	意味	例	結果
<code>==</code>	等しい	<code>10 == 10</code>	<code>True</code>
<code>!=</code>	等しくない	<code>10 != 5</code>	<code>True</code>
<code><</code>	より小さい	<code>10 < 20</code>	<code>True</code>
<code>></code>	より大きい	<code>10 > 20</code>	<code>False</code>
<code><=</code>	以下	<code>10 <= 10</code>	<code>True</code>

評価値

C言語: `0` / `1` (整数) $\Leftrightarrow$ Python: 結果は `True` / `False` (bool型)

さまざまな計算

ai_programming_1_02_03.py

```
1  # さまざまな計算
2
3  # 商と余り
4  print(17 // 5)
5  print(17 % 5)
6
7  # 硬貨の計算 (1350円を500円玉、100円玉、50円玉で表す)
8  total = 1350
9  coin_500 = total // 500
10 remaining = total % 500
11 coin_100 = remaining // 100
12 remaining = remaining % 100
13 coin_50 = remaining // 50
14 print("500円玉:", coin_500, "枚")
15 print("100円玉:", coin_100, "枚")
```

実行結果

```
3↵
2↵
500円玉: 2枚↵
100円玉: 3枚↵
50円玉: 1枚↵
1↵
6↵
True↵
True↵
False↵
True↵
```

- `//` と `%` で硬貨の枚数を計算
- `+=` で値を加算、比較演算子で真偽値を取得

さまざまな計算

ai_programming_1_02_03.py

```
5  print(17%5)
6
7  # 硬貨の計算 (1350円を500円玉、100円玉、50円玉で表す)
8  total = 1350
9  coin_500 = total // 500
10 remaining = total % 500
11 coin_100 = remaining // 100
12 remaining = remaining % 100
13 coin_50 = remaining // 50
14 print("500円玉:", coin_500, "枚")
15 print("100円玉:", coin_100, "枚")
16 print("50円玉:", coin_50, "枚")
17
18 # 複合代入演算子
19 count = 0
```

実行結果

```
3↵
2↵
500円玉: 2 枚↵
100円玉: 3 枚↵
50円玉: 1 枚↵
1↵
6↵
True↵
True↵
False↵
True↵
```

- `//` と `%` で硬貨の枚数を計算
- `+=` で値を加算、比較演算子で真偽値を取得

さまざまな計算

ai_programming_1_02_03.py

```
15 print("500円玉:", coin_500, "枚")
16 print("100円玉:", coin_100, "枚")
17
18 # 複合代入演算子
19 count = 0
20 count += 1
21 print(count)
22 count += 5
23 print(count)
24
25 # 比較演算子
26 print(10 == 10)
27 print(10 != 5)
28 print(10 > 20)
29 print(10 <= 10)
```

実行結果

```
3↵
2↵
500円玉: 2 枚↵
100円玉: 3 枚↵
50円玉: 1 枚↵
1↵
6↵
True↵
True↵
False↵
True↵
```

- `//` と `%` で硬貨の枚数を計算
- `+=` で値を加算、比較演算子で真偽値を取得

C言語との比較：演算子

Pythonにあり、C言語にないもの

- `**` (べき乗)
- `//` (整数除算の専用演算子)

C言語にあり、Pythonにないもの

- `++` / `--` (インクリメント・デクリメント)
- `? :` (三項演算子の記法が異なる)

C言語の `/` (整数同士) = Pythonの `//`
C言語の `/` (小数含む) = Pythonの `/`

まとめ

算術演算子

`+` `-` `*` `/`
`//` `%` `**`

複合代入

`+=` `-=` `*=` `/=`
`++` `--` はない

比較演算子

結果は `bool` 型
`True` / `False`

`//` と `%` は実用的な計算で頻出

入出力

print()関数の詳細

複数引数

```
>>> print("Hello", "World")  
Hello World  
>>> print(1, 2, 3)  
1 2 3
```

スペース区切りで表示

sep・endパラメータ

```
>>> print(1, 2, 3, sep=",")  
1, 2, 3  
>>> print("Hello", end="!")  
Hello!
```

区切り文字・末尾文字を指定

デフォルト: `sep=" "` (スペース)、`end="\n"` (改行)

input関数

input関数

```
変数 = input("プロンプト文字列")
```

- キーボードから入力を受け取る関数
- 入力値は常に **文字列 (str型)** として取得

```
>>> name = input("名前: ")
名前: Python
>>> print(name)
Python
>>> type(name)
<class 'str'>
```

input()と型変換の組み合わせ

数値入力

```
変数 = int(input("プロンプト"))
```

`input()` は文字列を返すため、数値計算には型変換が必要

型変換なし（エラー）

```
age = input("年齢：")  
print(age + 1) # TypeError!
```

`age` は "20"（文字列）のまま

型変換あり（正しい）

```
age = int(input("年齢："))  
print(age + 1) # 21
```

`age` は 20（整数）に変換済み

文字列の結合と表示

+ による文字列結合

```
>>> "Hello" + " " + "World"  
'Hello World'
```

数値との結合はエラー

```
>>> "年齢：" + 20  
TypeError!  
>>> "年齢：" + str(20)  
'年齢：20'
```

* による文字列の繰り返し

```
>>> "=" * 20  
'=================''
```

文字列と数値の結合には `str()` で変換が必要

f文字列（フォーマット済み文字列）

f文字列（f-string）

`f"テキスト・{式}・テキスト"`

- 文字列の中に変数や式を直接埋め込める
- 推奨される書式指定方法

```
name = "Python"  
age = 30  
print(f"{name}は{age}歳です。")  
print(f"来年は{age + 1}歳です。")
```

`str()` で変換する必要がなく、**簡潔で読みやすい**

format()

`format()` —
"テキスト・{}・テキスト".format(値)

- {} の位置に値が埋め込まれる
- f文字列より古い方法だが、読めるようにしておく

```
name = "Python"  
print("言語は{}です.".format(name))  
print("{}と{}".format("A", "B"))
```

新しいコードではf文字列を推奨

対話的なプログラム

ai_programming_1_02_04. py

```
1  # 対話的なプログラム
2
3  # 名前の入力
4  name = input("名前を入力してください: ")
5
6  # 年齢の入力 (文字列→整数に変換)
7  age = int(input("年齢を入力してください: "))
8
9  # f文字列で表示
10 print(f"{name}さんは{age}歳です。")
11
12 # 来年の年齢
13 next_age = age + 1
14 print(f"来年は{next_age}歳になります。")
```

実行結果

名前を入力してください: Python
年齢を入力してください: 20
Pythonさんは20歳です。
来年は21歳になります。

- `input()` で文字列・数値を入力
- `int()` で型変換、`f"..."` で表示

まとめ

print()

複数引数

`sep`, `end` で制御

input()

文字列として入力

数値は `int()` で
変換

f文字列

`f" {変数} "` で埋め込み

推奨の書式指定
方法

`input()` + `int()` + f文字列 で対話的なプログラムを作成

変数と代入の応用

多重代入

多重代入

```
a, b, c = 1, 2, 3
```

- 1行で複数の変数に同時に値を代入
- 左辺と右辺の数が一致する必要あり

```
>>> x, y = 10, 20
>>> print(x)
10
>>> print(y)
20
```

同じ値の代入: `a = b = c = 0` も可能

変数の入れ替え

C言語

```
int tmp = a;  
a = b;  
b = tmp;
```

一時変数が必要 (3行)

Python

```
a, b = b, a
```

多重代入で1行

```
>>> a, b = 10, 20  
>>> a, b = b, a  
>>> print(a, b)  
20 10
```

文字列操作の基礎

文字列の長さ: `len()`

```
>>> s = "Python"
>>> len(s)
6
```

インデックスアクセス

```
>>> s = "Python"
>>> s[0]
'P'
>>> s[5]
'n'
>>> s[-1]
'n'
```

インデックス	0	1	2	3	4	5
文字	P	y	t	h	o	n
負のインデックス	-6	-5	-4	-3	-2	-1

秒→時分秒変換プログラム

ai_programming_1_02_05.py

実行結果

```
1  # 秒→時分秒変換プログラム
2
3  # 秒数の入力
4  total_seconds = int(input("秒数を入力してください: "))
5
6  # 時間、分、秒の計算
7  hours = total_seconds // 3600
8  remaining = total_seconds % 3600
9  minutes = remaining // 60
10 seconds = remaining % 60
11
12 # 結果の表示
13 print(f"{total_seconds}秒は{hours}時間{minutes}分{seconds}秒です。")
```

秒数を入力してください: 3725
3725秒は1時間2分5秒です。

- `//` と `%` で時間・分・秒を計算
- `input()` + `int()` で数値入力

2数の平均計算プログラム

ai_programming_1_02_06.py

```
1  # 2数の平均計算プログラム
2
3  # 2つの整数の入力
4  a = int(input("1つ目の整数を入力してください: "))
5  b = int(input("2つ目の整数を入力してください: "))
6
7  # 平均の計算
8  average = (a + b) / 2
9
10 # 結果の表示
11 print(f"{a}と{b}の平均は{average}です。")
```

実行結果

1つ目の整数を入力してください: 80
2つ目の整数を入力してください: 70
80と70の平均は75.0です。

- 整数 / 整数 → 結果は float (75.0)
- input() + int() + f文字列 の総合

第2章のまとめ

変数と代入

変数名 = 値
動的型付け

データ型

int float
str bool

演算子

算術・比較
複合代入

入出力

print()
input()
f文字列

変数・型・演算子・入出力は **すべてのプログラムの基礎**

次章の予告：第3章 条件文

学ぶこと

- `if` 文による条件分岐
- `elif` / `else` による複数条件
- 比較演算子と論理演算子の活用

今回の知識との関係

- 比較演算子の結果（`True` / `False`）を条件判定に使用
- `input()` で入力した値に応じた処理の分岐

第2章で学んだ**比較演算子**が条件文の基礎になる